

Résumé des données utilisées:

Dans le contexte de ce projet nous avons utilisé diverses bibliothèques python permettant de nous simplifier la tâche ou bien d'accéder à des données que nous ne pourrions pas collecter sans. Nous avons donc utilisé pickle, networkx, osmnx et matplotlib.

La ville de Montréal et ces quartiers sont représentés sous forme de graphe pondéré, celui-ci a été récupéré de banque de donnée publique grâce à la bibliothèque osmnx, les graphes sont donc des objets de la bibliothèque networkx.

Le drone parcourt l'intégralité de la ville de Montréal, alors que les déneigeuses parcourent la ville quartier par quartier.

Nous avons effectué nos recherches grâce à jupyter notebook, cela nous permet d'être claire et explicite dans celle-ci et de proposer une relecture chronologique fluide.

PARTIE DRONE:

Contraintes par rapport à Montréal:

La ville de Montréal est un multigraphe non-orienté, pondéré et qui est connexe.

Contraintes par rapport aux drones:

Dans notre modélisation, nous n'avons pas attribué de vitesse au drone et n'avons pas considéré de contrainte de carburant etc... Nous avons aussi considéré que le drone n'avait pas la capacité de voler au-dessus de bâtiments et qu'il était donc forcé de suivre la route. Évidemment, étant donné que le drone vole, nous avons admis qu'il n'était pas contraint par le sens de la circulation.

(Hypothèses:)

Résumé progression:

I. Réflexion sur les différents algorithmes à adopter

Nos premières recherches nous ont amenés à considérer différents algorithmes, notamment Dijkstra, ou encore ceux liés au problème du facteur chinois. L'objectif était de générer un circuit eulérien efficace sur le graphe représentant la ville.

II. Tester les algorithmes sur des graphes aléatoires

Après plusieurs réflexions autour du coût algorithmique, nous avons retenu l'algorithme du facteur chinois. Nous avons lancé des tests sur des matrices d'adjacence générées aléatoirement, représentant divers graphes. Plusieurs problèmes sont apparus, notamment un temps de calcul trop élevé. Le circuit eulérien ne pouvait être généré que pour des graphes de 17 nœuds au maximum, ce qui est insuffisant pour un environnement urbain comme Montréal.

Conclusion : deux options s'offraient à nous : optimiser nos algorithmes manuellement ou recourir à des bibliothèques existantes permettant des implémentations plus efficaces.

III. Utiliser la bibliothèque *networkx*

La bibliothèque Python *networkx* répondait à nos besoins. Plusieurs de ses fonctions nous ont permis d'implémenter nos algorithmes à moindre coût.

Les résultats de ces fonctions se sont révélés concluants. Nous avons pu tester nos algorithmes sur des graphes aléatoires de plusieurs centaines de nœuds.

IV. Animer notre parcours

Une fois les graphes générés, rendus eulériens, et les circuits trouvés, nous avons souhaité visualiser le parcours via une interface graphique, afin de mieux le comprendre et éventuellement identifier des erreurs.

Pour cela, nous avons utilisé *matplotlib* pour afficher le graphe, en surlignant les arêtes parcourues en rouge.

Nous avons alors repéré un problème, notre algorithme est bête : il ne prend pas les meilleurs chemins en tenant compte des distances.

V. Le problème de la fonction *eulerize* de *networkx*

Un problème est alors apparu : la fonction *eulerize* ne prend pas en compte les poids des arêtes. Dans notre cas, ces poids étaient générés aléatoirement. Le circuit produit n'était donc pas optimal en termes de coût total.

Nous avons donc repris l'algorithme du facteur chinois que nous avons précédemment développé et l'avons adapté à *networkx* de façon à intégrer la notion de poids, ce qui permettait d'obtenir un parcours à la fois eulérien et moins coûteux.

Conclusion: Les algorithmes obtenus étaient à la fois plus rapides et plus logiques. Nous avons pu comparer la "distance" totale parcourue par les différents circuits, mesurée par la somme des poids des arêtes traversées. Les dernières versions de nos fonctions produisaient des parcours plus courts et donc plus optimisés. Nous avons aussi comparé la somme des distances du graphe à la distance parcourue par notre drone, et nous avons constaté que ces distances étaient relativement proches. Cela a donc confirmé l'efficacité de notre algorithme.

VI. Mise à l'échelle

Maintenant que nous avons une solution fonctionnelle pour notre drone, nous devons l'adapter à l'échelle de Montréal. Nous avons donc utilisé la bibliothèque *osmnx* pour récupérer la ville et les quartiers de Montréal sous forme de graphe. Une fois cela fait, il ne nous restait qu'à remplacer le graphe aléatoire précédemment généré.

Étant donné que l'algorithme peut prendre du temps lorsqu'il est lancé sur de grandes zones géographiques, nous avons décidé de sauvegarder la version eulérienne des quartiers de Montréal et de la ville en entier dans des fichiers afin de ne pas avoir à refaire l'ensemble des calculs à chaque fois que l'on souhaite faire la simulation du parcours du drone.

VII. Clustering

Euleriser tout Montréal étant trop coûteux, nous avons découpé le graphe en sous-graphes à l'aide de clustering. Chaque cluster est eulérisé séparément, puis les résultats sont fusionnés. Plus il y a de clusters, plus l'exécution est rapide, mais moins le circuit est optimal. Un compromis a été nécessaire pour choisir un nombre de clusters adapté.

VIII. Limites et contraintes de notre approche

Dans notre modélisation, nous n'avons pas attribué de vitesse au drone et n'avons pas considéré de contrainte de carburant etc... Nous avons aussi considéré que le drone n'avait pas la capacité de voler au-dessus de bâtiments et qu'il était donc forcé de suivre la route. Évidemment, étant donné que le drone vole, nous avons admis qu'il n'était pas contraint par le sens de la circulation.

PARTIE DÉNEIGEUSE:

Contraintes par rapport à Montréal:

Un quartier de Montréal est un multigraphe orienté, pondéré et qui n'est pas forcément fortement connexe.

Contraintes par rapport aux déneigeuses:

Les déneigeuses respectent le code de la route en respectant le sens des arêtes.

Une déneigeuse récolte la neige sur une arête si elle passe sur cette arête. Les calculs de parcours des déneigeuses sont calculés par quartier et non pas pour toute la ville d'un coup.

Hypothèses:

Nous avons supposé que le parcours correct du graphe est suffisant pour le respect du code de la route.

Nous avons originellement supposé que les quartiers de Montréal étaient fortement connexes mais cette hypothèse s'est avérée fausse ainsi que nuisible à notre progression.

Résumé progression:

Une fois que la liste des arêtes à déneiger a été obtenue, nous devons trouver l'itinéraire le plus efficace pour toutes les déneiger. Nous devons aussi proposer différents scénarios en fonction du type de déneigeuse (charrue en québécois), de leur nombre et d'autres variables comme la quantité de neige.

Pour représenter ce problème, nous travaillons avec un graphe pondéré orienté.

Nous avons tout d'abord cherché à établir un algorithme simple pouvant traverser une liste d'arêtes d'un graphe de manière relativement optimale.

Première implémentation:

Nous avons commencé par choisir une approche simple, aussi bien à comprendre qu'algorithmiquement. Elle consiste à placer une déneigeuse dans sur un point arbitraire du graphe, puis à aller vers l'arête la plus proche, la déneiger, puis aller à l'arête la plus proche, etc... Afin de savoir quelle est l'arête la plus proche, nous utilisons l'algorithme de Floyd-Warshall qui permet de connaître la distance minimale et le plus court chemin entre tous les couples de sommets du graphe. Cette approche nous a permis d'avoir des résultats assez concluants sur des graphes de tests générés aléatoirement.

En termes de temps de calcul, nous avons remarqué rapidement que la grande majorité du temps de calcul était prise par Floyd Warshall. Or Floyd-Warshall renvoie le même résultat quelque soit la répartition de la neige. Nous nous sommes donc dit qu'il serait judicieux de stocker cette donnée dans un fichier et de juste la load au moment de l'exécution.

Deuxième implémentation:

Nous avons donc ajusté nos fonctions au graphe obtenu, sauvegardant le résultat de Floyd Warshall en utilisant la bibliothèque pickle. Les calculs obtenus étaient satisfaisants lorsque nous nous sommes retrouvés face à un problème assez pénible. Le graphe fourni par osmnx contenait des arêtes impasse (ie. des rues sortantes du graphe à sens unique). Si enneigées, ces arêtes posent problème vu qu'une déneigeuse ne peut pas les emprunter sans être après bloquées dans un cul de sac. Pour palier à ce problème, nous avons décidé de ne pas considérer ces arêtes dans nos calculs.

Mais cela nous confronte donc à un nouveau problème, comment détecter ces arêtes ? Le problème vient du fait que le graph n'est tout simplement pas fortement connexe comme nous avons jusque là supposé, et ces arêtes ne sont pas forcément seules elles peuvent être des arêtes menant à un sous graphe d'où on ne peut pas sortir. En réalité il est relativement simple de détecter ces arêtes car elles font parties de composantes fortement connexe non maximum du graph, il est possible de calculer ces composantes grâce à networkx qui fournit une fonction faisant cela mais elle est trop lourde à exécuter et cela ne règle pas forcément le problème puisque de grandes parties du graphe sont ignorées, nous avons donc mis en place une simple heuristique pour détecter ces arêtes problématiques et les supprimer des arêtes à traverser, l'heuristique est basée sur l'accessibilité à tous les autres sommets à partir d'un sommet cible.

Optimisation et parallélisation:

Nous avons donc décidé d'utiliser la version précédente de l'algorithme de récolte de neige (qui ne peut donc gérer qu'une seule déneigeuse), cependant nous avons imaginé plusieurs moyens de pré-traiter l'information afin de pouvoir faire tourner plusieurs déneigeuses.

Nous avons décidé de prendre la liste d'arêtes à traverser et de la séparer en plusieurs listes contenant les différentes arêtes, la séparation doit être intelligente et les arêtes doivent être regroupées par proximité géographique. Pour cela nous utilisons l'information géographique des nœuds, nous calculons donc quatre points très séparés les uns des autres et utilisons le résultat de Floyd Warshall afin de déterminer pour chaque arête vers quel point il est le plus rapide d'aller, cela nous permet de déterminer à quel sous-liste l'arête appartient.

Calcul de métriques:

Nous avons ensuite créé des fonctions permettant de calculer les différentes métriques à partir du résultat de nos fonctions. Tout d'abord le kilométrage à partir du path, puis le coût du passage pour la ville, puis le temps pour tout déneiger. Nous avons utilisé ces fonctions pour trouver les scénarios les plus optimaux.